

Fetch och Promise i JavaScript

`fetch()` används i JavaScript för att **hämta resurser**, till exempel data från ett API eller innehåll från en annan fil.

En viktig sak med `fetch()` är att hämtningen sker **asynkront**. JavaScript behöver alltså inte stanna och vänta medan exempelvis en fil laddas ner.

1. Ett enkelt exempel med fetch

Vi vill hämta innehållet från filen `header.html`:

```
fetch("header.html")
  .then(response => response.text())
  .then(html => {
    console.log(html);
  });
```

Här händer flera saker:

1. `fetch()` börjar hämta filen.
 2. `fetch()` returnerar ett **Promise**.
 3. När hämtningen är klar körs den första `.then()`.
 4. `response.text()` läser innehållet som text.
 5. När texten är färdigläst körs nästa `.then()`.
-

Vad är ett Promise?

Ett **Promise** kan beskrivas som ett löfte om att ett resultat kommer att finnas **senare**.

Tänk:

“Jag har inte resultatet ännu, men jag lovar att meddela dig när det är klart.”

Ett Promise kan befinna sig i tre olika tillstånd:

Tillstånd	Betydelse
pending	Vi väntar fortfarande på resultatet
fulfilled	Operationen lyckades
rejected	Operationen misslyckades

När vi skriver:

```
const resultat = fetch("header.html");
```

innehåller resultat alltså **inte HTML-koden**.

Det innehåller ett **Promise**.

2. .then() väntar på ett Promise

För att göra något när ett Promise är färdigt kan vi använda .then():

```
fetch("header.html")
  .then(response => {
    console.log("Filen har hämtats!");
  });
```

Funktionen i .then() körs först när fetch() har fått ett svar.

Det gör att resten av JavaScript-programmet kan fortsätta köras under tiden.

3. Response är inte samma sak som innehållet

När fetch() är klar får vi ett **Response-objekt**:

```
fetch("header.html")
  .then(response => {
    console.log(response);
  });
```

Det innehåller bland annat information om HTTP-svaret.

För att få själva texten behöver vi använda:

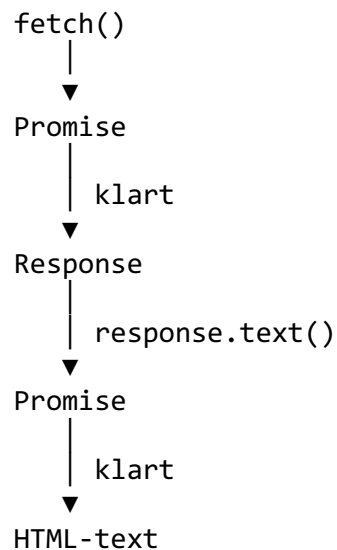
```
response.text()
```

Men även response.text() är asynkron och returnerar därför ett **Promise**.

Därför får vi två .then():

```
fetch("header.html")
  .then(response => response.text())
  .then(html => {
    console.log(html);
  });
```

Man kan tänka på flödet så här:



4. Lägga in HTML på sidan

Om sidan innehåller:

```
<header id="header"></header>
```

kan vi hämta header.html och placera innehållet där:

```
fetch("header.html")
  .then(response => response.text())
  .then(html => {
    document.querySelector("#header").innerHTML = html;
  });
```

På så sätt kan samma header.html användas på flera sidor.

Exempel på filer:

```
webbplats/
├── index.html
├── kontakt.html
├── om.html
├── script.js
├── components/
│   ├── header.html
│   └── footer.html
```

JavaScript:

```
fetch("components/header.html")
  .then(response => response.text())
  .then(html => {
    document.querySelector("#header").innerHTML = html;
  });
```

5. Vad händer om något går fel?

Vi kan använda `.catch()` för att fånga vissa fel:

```
fetch("header.html")
  .then(response => response.text())
  .then(html => {
    document.querySelector("#header").innerHTML = html;
  })
  .catch(error => {
    console.error("Något gick fel:", error);
  });
```

Det är dock viktigt att känna till att `fetch()` normalt **inte rejectar sitt Promise bara för att servern svarar med exempelvis 404**.

Därför bör vi även kontrollera svaret:

```
fetch("header.html")
  .then(response => {
    if (!response.ok) {
      throw new Error("HTTP-fel: " + response.status);
    }

    return response.text();
  })
  .then(html => {
    document.querySelector("#header").innerHTML = html;
  })
  .catch(error => {
    console.error("Kunde inte hämta filen:", error);
  });
```

6. Promise-kedjan

En viktig egenskap hos `.then()` är att den själv returnerar ett Promise.

Därför kan flera operationer kopplas ihop:

```
fetch("header.html")
  .then(response => response.text())
  .then(html => {
    document.querySelector("#header").innerHTML = html;
  })
  .then(() => {
    console.log("Headern är färdig!");
  });
```

Detta brukar kallas en **Promise chain**.

```
fetch()
  ↓
.then()
  ↓
.then()
  ↓
.then()
```

Varje steg väntar på resultatet från föregående steg.

7. Samma kod med async/await

Promises kan också hanteras med `async` och `await`.

Samma exempel kan skrivas:

```
async function loadHeader() {
  const response = await fetch("header.html");
  const html = await response.text();

  document.querySelector("#header").innerHTML = html;
}

loadHeader();
```

`await` betyder ungefär:

Vänta här tills detta Promise är färdigt innan funktionen fortsätter.

Det gör ofta asynkron kod lättare att läsa.

`.then()` jämfört med `await`

Med `.then()`:

```
fetch("header.html")
  .then(response => response.text())
  .then(html => {
    document.querySelector("#header").innerHTML = html;
  });
```

Med await:

```
async function loadHeader() {
  const response = await fetch("header.html");
  const html = await response.text();

  document.querySelector("#header").innerHTML = html;
}
```

Båda bygger på **Promises**.

async/await är alltså inte ett alternativ till Promise – det är ett annat sätt att **arbeta med Promises**.

8. Viktigt när du testar fetch

fetch() bör normalt testas via en **webbserver**.

Att dubbelklicka på exempelvis:

index.html

och öppna sidan som:

file:///...

kan orsaka problem när sidan försöker hämta andra filer.

Använd i stället exempelvis en lokal utvecklingsserver så att sidan öppnas via en adress i stil med:

http://localhost:...

Kom ihåg

Fetch

```
fetch("fil.html")
```

startar en asynkron hämtning och returnerar ett **Promise**.

Promise

representerar ett resultat som kanske inte finns ännu.

Then

`.then(...)`

anger vad som ska hända när ett Promise är färdigt.

Catch

`.catch(...)`

kan hantera fel när ett Promise blir rejected.

Await

```
const response = await fetch("fil.html");
```

ger ett mer sekventiellt sätt att skriva kod som fortfarande bygger på Promises.

Kort sammanfattning

```
fetch()
  ↓
Promise
  ↓
Response
  ↓
response.text()
  ↓
Promise
  ↓
Text/data
  ↓
Använd resultatet
```

Grundidén: JavaScript behöver inte stå still och vänta på att nätverket ska bli färdigt. `fetch()` startar arbetet och ger tillbaka ett Promise som gör att vi kan hantera resultatet när det kommer.